

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки



Лабораторна робота
З дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав: ст.гр IX-22

Брона Марко

Прийняв:

Гордійчук-Бублівська О.В

Львів – 2026

Мета:

Ознайомлення з основами моделювання телекомунікаційних мереж. Вивчення використання об'єктно-орієнтованого програмування для моделювання мережі. Моделювання різних топологій мереж (зірка, кільцева, деревоподібна, сіткова (mesh), комбінована тощо). Симуляція передачі пакетів між вузлами та маршрутизаторами з використанням TCP та UDP. Вимірювання продуктивності мережі: швидкість передачі, статистика втрат, затримки передачі. Побудова візуалізації для графічного відображення структури та роботи мережі.

Послідовність виконання:

- *Вибір топології*
- *Кільцева (Ring)*
- *Створення класів вузлів мережі*
- *Визначте структуру класів Node, Router для відображення мережеских вузлів.*
- *Додайте методи для встановлення з'єднань між вузлами.*
- *Реалізація передавання пакетів*
- *Створіть клас Packet, що міститиме дані про відправника, отримувача, розмір пакета.*
- *Додайте симуляцію випадкової втрати пакетів (10-15% шанс втрати).*
- *Розробка моделей для TCP та UDP*
- *Реалізуйте класи TCPProtocol і UDPProtocol, які оброблятимуть пакети відповідно до правил кожного протоколу.*
- *Створення асинхронного механізму моделювання*
- *Використайте асунсіо для моделювання мережевого трафіку.*
- *Заплануйте події передачі пакетів з випадковими затримками.*
- *Вимірювання продуктивності*
- *Обчисліть статистику для:*
- *Часу передачі пакетів*
- *Кількості втрачених пакетів*
- *Пропускної здатності мережі*
- *Візуалізація мережі*

- Використайте `networkx` для створення графічного представлення обраної топології.
- Позначте активні зв'язки між вузлами.
- Аналіз продуктивності
- Порівняйте результати для різних топологій.
- Зробіть висновки про ефективність мережевої топології на основі отриманих даних.

```

1
2 import asyncio
3 import random
4 import time
5 import networkx as nx
6 import matplotlib.pyplot as plt
7 import matplotlib.patches as mpatches
8
9 class Node:
10     """Вузол мережі (комп'ютер / пристрій)."""
11
12     def __init__(self, name: str):
13         self.name = name
14         self.connections: list["Node"] = []
15
16     def connect(self, node: "Node"):
17         """Встановлення двостороннього з'єднання між вузлами."""
18         if node not in self.connections:
19             self.connections.append(node)
20         if self not in node.connections:
21             node.connections.append(self)
22
23     async def send(self, packet: "Packet", network: "Network"):
24         """Асинхронна передача пакета з випадковою затримкою та імовірністю втрати.
25         await asyncio.sleep(random.uniform(0.02, 0.15)) # затримка передачі
26         if random.random() < network.loss_rate:
27             network.packets_lost += 1
28             if network.verbose:
29                 print(f" [ВІПАТА] Пакет {packet.src.name} → {packet.dest.name} "
30                       f"({packet.protocol}) втрачено на {self.name}")
31             return
32         await self.forward(packet, network)
33
34     async def forward(self, packet: "Packet", network: "Network"):
35         """Маршрутизація пакета до вузла-отримувача."""
36         if self == packet.dest:
37             network.packets_delivered += 1
38             if network.verbose:
39                 print(f" [OK] Пакет {packet.src.name} → {packet.dest.name} "
40                       f"({packet.protocol}, {packet.size} байт) доставлено")
41             return
42         if self in packet.visited:
43             return
44         packet.visited.append(self)
45         for node in self.connections:
46             if node not in packet.visited:
47                 await node.send(packet, network)
48
49     def __repr__(self):
50         return f"Node({self.name})"

```

```

C > Users > Marko > Downloads > 📁 la62_OOP_3.py
9 class Node:
51
52
53 class Router(Node):
54     """Маршрутизатор – центральний вузол мережі."""
55
56     def __repr__(self):
57         return f"Router({self.name})"
58
59
60 class Packet:
61     """Мережевий пакет з інформацією про відправника, отримувача, протокол."""
62
63     def __init__(self, src: Node, dest: Node, size: int, protocol: str):
64         self.src = src
65         self.dest = dest
66         self.size = size # розмір у байтах
67         self.protocol = protocol
68         self.visited: list[Node] = []
69
70     def __repr__(self):
71         return (f"Packet(src={self.src.name}, dest={self.dest.name}, "
72               f"size={self.size}, protocol={self.protocol})")
73
74
75 class TCPProtocol:
76     """
77     TCP – надійна передача з підтвердженням.
78     Великий розмір пакета (200-500 байт), імітація підтвердження.
79     """
80     name = "TCP"
81
82     @staticmethod
83     async def transmit(src: Node, dest: Node, network: "Network"):
84         packet = Packet(src, dest, random.randint(200, 500), "TCP")
85         # Імітація трьохстороннього handshake (додаткова затримка)
86         await asyncio.sleep(random.uniform(0.01, 0.05))
87         await src.send(packet, network)
88
89
90 class UDPProtocol:
91     """
92     UDP – швидка передача без підтвердження.
93     Малий розмір пакета (50-200 байт), без додаткових затримок.
94     """
95     name = "UDP"
96
97     @staticmethod
98     async def transmit(src: Node, dest: Node, network: "Network"):
99         packet = Packet(src, dest, random.randint(50, 200), "UDP")
100         await src.send(packet, network)

```

```

C > Users > Marko > Downloads > 📁 la62_OOP_3.py
143 class Network:
144
145     def analyze(self):
146         avg_time = (self.total_time / self.packets_sent)
147         loss_pct = (self.packets_lost / self.packets_sent) * 100
148         delivered = self.packets_sent - self.packets_lost
149         bandwidth = delivered / self.total_time if self.total_time > 0 else 0
150
151         print(f"\n--- Результати ({self.topology_name}) ---")
152         print(f" Відправлено пакетів : {self.packets_sent}")
153         print(f" Доставлено пакетів : {delivered}")
154         print(f" Втрачено пакетів : {self.packets_lost} ({loss_pct:1f}%)")
155         print(f" Середній час передачі: {avg_time:4f} с")
156         print(f" Пропускна здатність : {bandwidth:2f} пак/с")
157         print(f" Загальний час : {self.total_time:4f} с")
158
159         return {
160             "topology": self.topology_name,
161             "sent": self.packets_sent,
162             "delivered": delivered,
163             "lost": self.packets_lost,
164             "loss_pct": loss_pct,
165             "avg_time": avg_time,
166             "bandwidth": bandwidth,
167         }
168
169     def visualize(self, ax=None, title: str = None):
170         """Любуємося графічним представленням топології мережі."""
171         G = nx.Graph()
172         node_colors = []
173         node_labels = []
174
175         for node in self.nodes:
176             G.add_node(node.name)
177             node_labels[node.name] = node.name
178             if isinstance(node, Router):
179                 node_colors.append("#FF6B6B") # червоний – маршрутизатор
180             else:
181                 node_colors.append("#74B9FF") # синій – ПК
182
183         for node in self.nodes:
184             for conn in node.connections:
185                 if not G.has_edge(node.name, conn.name):
186                     G.add_edge(node.name, conn.name)
187
188         pos = self._layout(G)
189
190         show = ax is None
191         if show:
192             fig, ax = plt.subplots(figsize=(8, 6))
193
194         nx.draw_networkx_nodes(G, pos, ax=ax, node_color=node_colors,

```

```

C > Users > Marko > Downloads > 📁 la62_OOP_3.py
103 class Network:
104
105     def visualize(self, ax=None, title: str = None):
106         if show:
107             fig, ax = plt.subplots(figsize=(8, 6))
108
109         nx.draw_networkx_nodes(G, pos, ax=ax, node_color=node_colors,
110                               node_size=2800, alpha=0.9)
111         nx.draw_networkx_labels(G, pos, ax=ax, labels=node_labels,
112                                font_size=9, font_weight="bold")
113         nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#636E72",
114                               width=2, alpha=0.7)
115
116         legend_handles = [
117             mpatches.Patch(color="#FF6B6B", label="Router"),
118             mpatches.Patch(color="#74B9FF", label="PC / Node"),
119         ]
120         ax.legend(handles=legend_handles, loc="upper right", fontsize=9)
121         ax.set_title(title or self.topology_name, fontsize=13, fontweight="bold")
122         ax.axis("off")
123
124         if show:
125             plt.tight_layout()
126             plt.show()
127
128     def _layout(self, G):
129         """Алгоритм розміщення вузлів (перевизначається в підкласах)."""
130         return nx.spring_layout(G, seed=42)
131
132 class StarNetwork(Network):
133
134     topology_name = "Зіркова (Star)"
135
136     def __init__(self, pc_count: int = 5, **kwargs):
137         super().__init__(**kwargs)
138         self._build(pc_count)
139
140     def _build(self, pc_count: int):
141         router = Router("Router")
142         pcs = [Node(f"PC{i}") for i in range(1, pc_count + 1)]
143         self.nodes = {router} + pcs
144         for pc in pcs:
145             router.connect(pc)
146         print(f"[Star] Побудовано: 1 роутер + {pc_count} ПК")
147
148     def _layout(self, G):
149         pos = {}
150         router_name = next(n.name for n in self.nodes if isinstance(n, Router))
151         pos[router_name] = (0.0, 0.0)
152         pcs = [n for n in self.nodes if not isinstance(n, Router)]
153         angle_step = 2 * 3.14159 / len(pcs)

```

```
Ла62_ООП_3.py
C:\> Users\Marko\Downloads\Ла62_ООП_3.py
223 class StarNetwork(Network):
224     def _layout(self, G):
225         pos[router_name] = (0.0, 0.0)
226         pcs = [n for n in self.nodes if not isinstance(n, Router)]
227         angle_step = 2 * 3.14159 / len(pcs)
228         for i, pc in enumerate(pcs):
229             angle = i * angle_step
230             import math
231             pos[pc.name] = (math.cos(angle), math.sin(angle))
232         return pos
233
234 class RingNetwork(Network):
235     topology_name = "Кільцева (Ring)"
236
237     def __init__(self, node_count: int = 6, **kwargs):
238         super().__init__(**kwargs)
239         self._build(node_count)
240
241     def _build(self, node_count: int):
242         nodes = [Node(f"PC{i}") for i in range(1, node_count + 1)]
243         self.nodes = nodes
244         for i in range(len(nodes)):
245             nodes[i].connect(nodes[(i + 1) % len(nodes)])
246         print(f"[Ring] Побудовано: {node_count} вузлів у кільці")
247
248     def _layout(self, G):
249         return nx.circular_layout(G)
250
251 def compare_results(results: list[dict]):
252     """Виводить порівняльну таблицю результатів симуляцій."""
253     print(f"\n{'='*65}")
254     print(f" ПОРІВНЯЛЬНИЙ АНАЛІЗ ТОПОЛОГІЙ")
255     print(f"{'='*65}")
256     header = f"{'Топологія':<22} {'Прот.':<6} {'Відпр.':<8} {'Достав.':<8} "
257     header += f"{'Втрати':<8} {'Серед.час':<11} {'Пропуск.':<10}"
258     print(header)
259     print("-" * 65)
260     for r in results:
261         row = (f"{r['topology']:<22} {r['protocol']:<6} {r['sent']:<8} "
262               f"{r['delivered']:<8} {r['loss_pct']:<7.1f}% "
263               f"{r['avg_time']:<11.4f} {r['bandwidth']:<2f} пак/с")
264         print(row)
265     print("-" * 65)
266
267 def plot_comparison(results: list[dict]):
268     """Побудова діаграм для порівняння топологій."""
269     labels = [f"{r['topology']}\n{r['protocol']}" for r in results]
270     loss_vals = [r['loss_pct'] for r in results]
271     bw_vals = [r['bandwidth'] for r in results]
272     time_vals = [r['avg_time'] * 1000 for r in results] # у мс
```

```
Ла62_ООП_3.py
C:\> Users\Marko\Downloads\Ла62_ООП_3.py
223 async def main():
224     res = star_net.analyze()
225     res["protocol"] = "TCP"
226     all_results.append(res)
227
228     # UDP
229     await star_net.simulate(UDPProtocol, packets=PACKETS)
230     res = star_net.analyze()
231     res["protocol"] = "UDP"
232     all_results.append(res)
233
234     # --- 2. Кільцева топологія ---
235     print("\n" + "#" * 55)
236     print("# КІЛЬЦЕВА ТОПОЛОГІЯ (Ring)                #")
237     print("#" * 55)
238
239     ring_net = RingNetwork(node_count=6, loss_rate=LOSS_RATE, verbose=VERBOSE)
240
241     # TCP
242     await ring_net.simulate(TCPProtocol, packets=PACKETS)
243     res = ring_net.analyze()
244     res["protocol"] = "TCP"
245     all_results.append(res)
246
247     # UDP
248     await ring_net.simulate(UDPProtocol, packets=PACKETS)
249     res = ring_net.analyze()
250     res["protocol"] = "UDP"
251     all_results.append(res)
252
253     # --- 3. Порівняльна таблиця ---
254     compare_results(all_results)
255
256     # --- 4. Візуалізація топологій поруч ---
257     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
258     fig.suptitle("Топології мережі", fontsize=15, fontweight="bold")
259     star_net.visualize(ax=axes[0], title="Зіркова топологія (Star)")
260     ring_net.visualize(ax=axes[1], title="Кільцева топологія (Ring)")
261     plt.tight_layout()
262     plt.show()
263
264     # --- 5. Порівняльні діаграми ---
265     plot_comparison(all_results)
266
267     # --- 6. Висновки ---
268     print("\n" + "="*55)
269     print(" ВИСНОВКИ")
270     print("="*55)
271     print("""
```

```
Ла62_ООП_3.py
C:\> Users\Marko\Downloads\Ла62_ООП_3.py
285 def plot_comparison(results: list[dict]):
286     time_vals = [r['avg_time'] * 1000 for r in results] # у мс
287
288     colors = ["#FF6B6B", "#FDCB6E", "#74B9FF", "#55EFC4"]
289
290     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
291     fig.suptitle("Порівняльний аналіз топологій (Star vs Ring)", fontsize=14, fontweight="bold")
292
293     # Втрати пакетів
294     axes[0].bar(labels, loss_vals, color=colors[:len(labels)], edgecolor="gray")
295     axes[0].set_title("Втрати пакетів (%)")
296     axes[0].set_ylabel("%")
297     axes[0].set_ylim(0, max(loss_vals) * 1.4 + 5)
298     for i, v in enumerate(loss_vals):
299         axes[0].text(i, v + 0.3, f"{v:.1f}%", ha="center", fontsize=9)
300
301     # Пропускна здатність
302     axes[1].bar(labels, bw_vals, color=colors[:len(labels)], edgecolor="gray")
303     axes[1].set_title("Пропускна здатність (пак/с)")
304     axes[1].set_ylabel("пак/с")
305     for i, v in enumerate(bw_vals):
306         axes[1].text(i, v + 0.1, f"{v:.2f}", ha="center", fontsize=9)
307
308     # Середній час передачі
309     axes[2].bar(labels, time_vals, color=colors[:len(labels)], edgecolor="gray")
310     axes[2].set_title("Середній час передачі (мс)")
311     axes[2].set_ylabel("мс")
312     for i, v in enumerate(time_vals):
313         axes[2].text(i, v + 0.5, f"{v:.1f}", ha="center", fontsize=9)
314
315     plt.tight_layout()
316     plt.show()
317
318 async def main():
319     PACKETS = 10 # кількість пакетів у кожній симуляції
320     LOSS_RATE = 0.12 # 12% втрат (однакові для обох мереж)
321     VERBOSE = True # детальне виведення
322
323     all_results = []
324
325     # --- 1. Зіркова топологія ---
326     print("\n" + "#" * 55)
327     print("# ЗІРКОВА ТОПОЛОГІЯ (Star)                #")
328     print("#" * 55)
329
330     star_net = StarNetwork(pc_count=5, loss_rate=LOSS_RATE, verbose=VERBOSE)
331
332     # TCP
333     await star_net.simulate(TCPProtocol, packets=PACKETS)
334     res = star_net.analyze()
335
336     # UDP
337     await star_net.simulate(UDPProtocol, packets=PACKETS)
338     res = star_net.analyze()
339
340     # --- Результати (Зіркова (Star)) ---
341     Відправлено пакетів : 10
342     Доставлено пакетів : 5
343     Втрачено пакетів : 5 (50.0%)
344     Середній час передачі: 0.0585 с
345     Пропускна здатність : 8.55 пак/с
346     Загальний час : 0.5848 с
347
348     # --- Результати (Кільцева (Ring)) ---
349     Відправлено пакетів : 10
350     Доставлено пакетів : 5
351     Втрачено пакетів : 5 (50.0%)
352     Середній час передачі: 0.0734 с
353     Пропускна здатність : 8.55 пак/с
354     Загальний час : 0.5848 с
355
356     # --- Результати (Зіркова (Star)) ---
357     Відправлено пакетів : 10
358     Доставлено пакетів : 5
359     Втрачено пакетів : 5 (50.0%)
360     Середній час передачі: 0.0734 с
361     Пропускна здатність : 8.55 пак/с
362     Загальний час : 0.5848 с
363
364     # --- Результати (Кільцева (Ring)) ---
365     Відправлено пакетів : 10
366     Доставлено пакетів : 5
367     Втрачено пакетів : 5 (50.0%)
368     Середній час передачі: 0.0734 с
369     Пропускна здатність : 8.55 пак/с
370     Загальний час : 0.5848 с
```

```
python Ла62_ООП.py
#####
# ЗІРКОВА ТОПОЛОГІЯ (Star)                #
#####
[Star] Побудовано: 1 роутер + 5 ПК
#####
Топологія: Зіркова (Star) | Протокол: TCP
Кількість пакетів: 10 | Рівень втрат: 12.0%
#####
[OK] Пакет PC3 -> Router (TCP, 386 байт) доставлено
[OK] Пакет Router -> PC1 (TCP, 423 байт) доставлено
[ВТРАТА] Пакет PC2 -> PC1 (TCP) втрачено на Router
[ВТРАТА] Пакет PC3 -> PC2 (TCP) втрачено на PC2
[ВТРАТА] Пакет PC5 -> PC2 (TCP) втрачено на PC1
[OK] Пакет PC1 -> PC2 (TCP, 251 байт) доставлено
[ВТРАТА] Пакет PC3 -> PC4 (TCP) втрачено на PC4
[OK] Пакет PC3 -> PC5 (TCP, 276 байт) доставлено
[OK] Пакет PC5 -> PC2 (TCP, 376 байт) доставлено
[ВТРАТА] Пакет PC1 -> PC5 (TCP) втрачено на PC4
[OK] Пакет PC1 -> PC5 (TCP, 427 байт) доставлено
[OK] Пакет PC5 -> PC4 (TCP, 425 байт) доставлено
#####
--- Результати (Зіркова (Star)) ---
Відправлено пакетів : 10
Доставлено пакетів : 5
Втрачено пакетів : 5 (50.0%)
Середній час передачі: 0.0585 с
Пропускна здатність : 8.55 пак/с
Загальний час : 0.5848 с
#####
Топологія: Зіркова (Star) | Протокол: UDP
Кількість пакетів: 10 | Рівень втрат: 12.0%
#####
[ВТРАТА] Пакет PC1 -> PC5 (UDP) втрачено на PC1
[OK] Пакет PC1 -> Router (UDP, 184 байт) доставлено
[OK] Пакет PC2 -> Router (UDP, 114 байт) доставлено
[OK] Пакет PC4 -> PC1 (UDP, 83 байт) доставлено
[OK] Пакет PC5 -> PC1 (UDP, 159 байт) доставлено
[OK] Пакет PC5 -> PC2 (UDP, 186 байт) доставлено
[ВТРАТА] Пакет PC5 -> PC4 (UDP) втрачено на PC1
[OK] Пакет PC1 -> PC5 (UDP, 190 байт) доставлено
[ВТРАТА] Пакет PC5 -> PC2 (UDP) втрачено на PC3
[ВТРАТА] Пакет PC5 -> PC3 (UDP) втрачено на PC3
[OK] Пакет PC1 -> PC5 (UDP, 87 байт) доставлено
[ВТРАТА] Пакет PC5 -> PC4 (UDP) втрачено на PC2
[OK] Пакет PC5 -> PC4 (UDP, 180 байт) доставлено
#####
--- Результати (Зіркова (Star)) ---
Відправлено пакетів : 10
Доставлено пакетів : 5
Втрачено пакетів : 5 (50.0%)
Середній час передачі: 0.0734 с
Пропускна здатність : 8.55 пак/с
Загальний час : 0.5848 с
```

```
Go Run ... Search

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Marko> python Ла62_00П.py
[ВТРАТА] Пакет PC5 → PC4 (UDP) втрачено на PC2
[OK] Пакет PC5 → PC4 (UDP, 180 байт) доставлено

--- Результати (Зіркова (Star)) ---
Відправлено пакетів : 10
Доставлено пакетів : 5
Втрачено пакетів : 5 (50.0%)
Середній час передачі: 0.0734 с
Пропускна здатність : 6.81 пак/с
Загальний час : 0.7343 с

#####
# КІЛЬЦЕВА ТОПОЛОГІЯ (Ring) #
#####
[Ring] Побудовано: 6 вузлів у кільці

=====
Топологія: Кільцева (Ring) | Протокол: TCP
Кількість пакетів: 10 | Рівень втрат: 12.0%
=====

[OK] Пакет PC3 → PC2 (TCP, 282 байт) доставлено
[OK] Пакет PC3 → PC2 (TCP, 312 байт) доставлено
[OK] Пакет PC1 → PC2 (TCP, 401 байт) доставлено
[OK] Пакет PC3 → PC1 (TCP, 368 байт) доставлено
[ВТРАТА] Пакет PC2 → PC3 (TCP) втрачено на PC5
[OK] Пакет PC1 → PC3 (TCP, 287 байт) доставлено
[ВТРАТА] Пакет PC3 → PC2 (TCP) втрачено на PC4
[OK] Пакет PC2 → PC3 (TCP, 398 байт) доставлено
[OK] Пакет PC5 → PC3 (TCP, 294 байт) доставлено
[ВТРАТА] Пакет PC1 → PC2 (TCP) втрачено на PC6
[ВТРАТА] Пакет PC1 → PC3 (TCP) втрачено на PC3
[OK] Пакет PC2 → PC4 (TCP, 355 байт) доставлено
[OK] Пакет PC3 → PC1 (TCP, 368 байт) доставлено
[ВТРАТА] Пакет PC1 → PC3 (TCP) втрачено на PC3
[OK] Пакет PC5 → PC3 (TCP, 294 байт) доставлено
[ВТРАТА] Пакет PC3 → PC4 (TCP) втрачено на PC4
[OK] Пакет PC2 → PC4 (TCP, 355 байт) доставлено
[OK] Пакет PC3 → PC2 (TCP, 312 байт) доставлено
[OK] Пакет PC3 → PC4 (TCP, 453 байт) доставлено
[OK] Пакет PC1 → PC3 (TCP, 416 байт) доставлено

--- Результати (Кільцева (Ring)) ---
Відправлено пакетів : 10
Доставлено пакетів : 4
Втрачено пакетів : 6 (60.0%)
Середній час передачі: 0.0817 с
Пропускна здатність : 4.89 пак/с
Загальний час : 0.8173 с

=====
Топологія: Кільцева (Ring) | Протокол: UDP
Кількість пакетів: 10 | Рівень втрат: 12.0%
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

loads

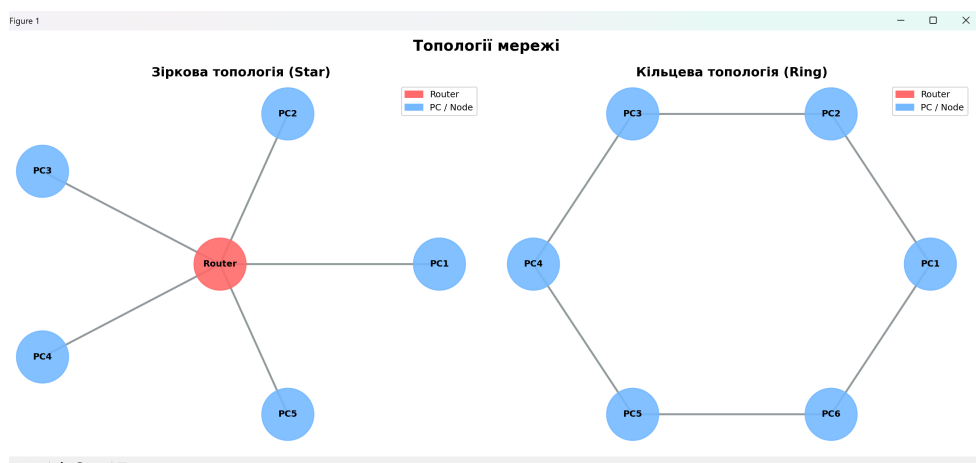
PS C:\Users\Marko> python Ла62_00П.py
Пропускна здатність : 4.89 пак/с
Загальний час : 0.8173 с

=====
Топологія: Кільцева (Ring) | Протокол: UDP
Кількість пакетів: 10 | Рівень втрат: 12.0%
=====

[ВТРАТА] Пакет PC2 → PC4 (UDP) втрачено на PC2
[OK] Пакет PC5 → PC3 (UDP, 113 байт) доставлено
[ВТРАТА] Пакет PC3 → PC1 (UDP) втрачено на PC2
[OK] Пакет PC6 → PC5 (UDP, 161 байт) доставлено
[OK] Пакет PC4 → PC6 (UDP, 100 байт) доставлено
[ВТРАТА] Пакет PC4 → PC5 (UDP) втрачено на PC2
[OK] Пакет PC5 → PC2 (UDP, 63 байт) доставлено
[OK] Пакет PC4 → PC6 (UDP, 132 байт) доставлено
[OK] Пакет PC3 → PC5 (UDP, 193 байт) доставлено
[OK] Пакет PC4 → PC5 (UDP, 130 байт) доставлено
[ВТРАТА] Пакет PC4 → PC6 (UDP) втрачено на PC6
[OK] Пакет PC3 → PC5 (UDP, 193 байт) доставлено
[OK] Пакет PC4 → PC6 (UDP, 100 байт) доставлено
[OK] Пакет PC5 → PC2 (UDP, 63 байт) доставлено
[OK] Пакет PC5 → PC3 (UDP, 113 байт) доставлено
[OK] Пакет PC3 → PC1 (UDP, 73 байт) доставлено
[OK] Пакет PC6 → PC5 (UDP, 161 байт) доставлено
[OK] Пакет PC3 → PC4 (UDP, 52 байт) доставлено
[OK] Пакет PC3 → PC4 (UDP, 52 байт) доставлено

--- Результати (Кільцева (Ring)) ---
Відправлено пакетів : 10
Доставлено пакетів : 6
Втрачено пакетів : 4 (40.0%)
Середній час передачі: 0.0816 с
Пропускна здатність : 7.36 пак/с
Загальний час : 0.8157 с

=====
ПОРІВНЯЛЬНИЙ АНАЛІЗ ТОПОЛОГІЙ
=====
Топологія Прот. Відпр. Достав. Втрати Серед.час Пропуск.
-----
Зіркова (Star) TCP 10 5 50.0 % 0.0585 8.55 пак/с
Зіркова (Star) UDP 10 5 50.0 % 0.0734 6.81 пак/с
Кільцева (Ring) TCP 10 4 60.0 % 0.0817 4.89 пак/с
Кільцева (Ring) UDP 10 6 40.0 % 0.0816 7.36 пак/с
=====
```



Висновок:

У ході лабораторної роботи я ознайомився з теоретичними основами моделювання телекомунікаційних мереж. Практична частина роботи була присвячена застосуванню принципів об'єктно-орієнтованого програмування для побудови програмних моделей мережових структур